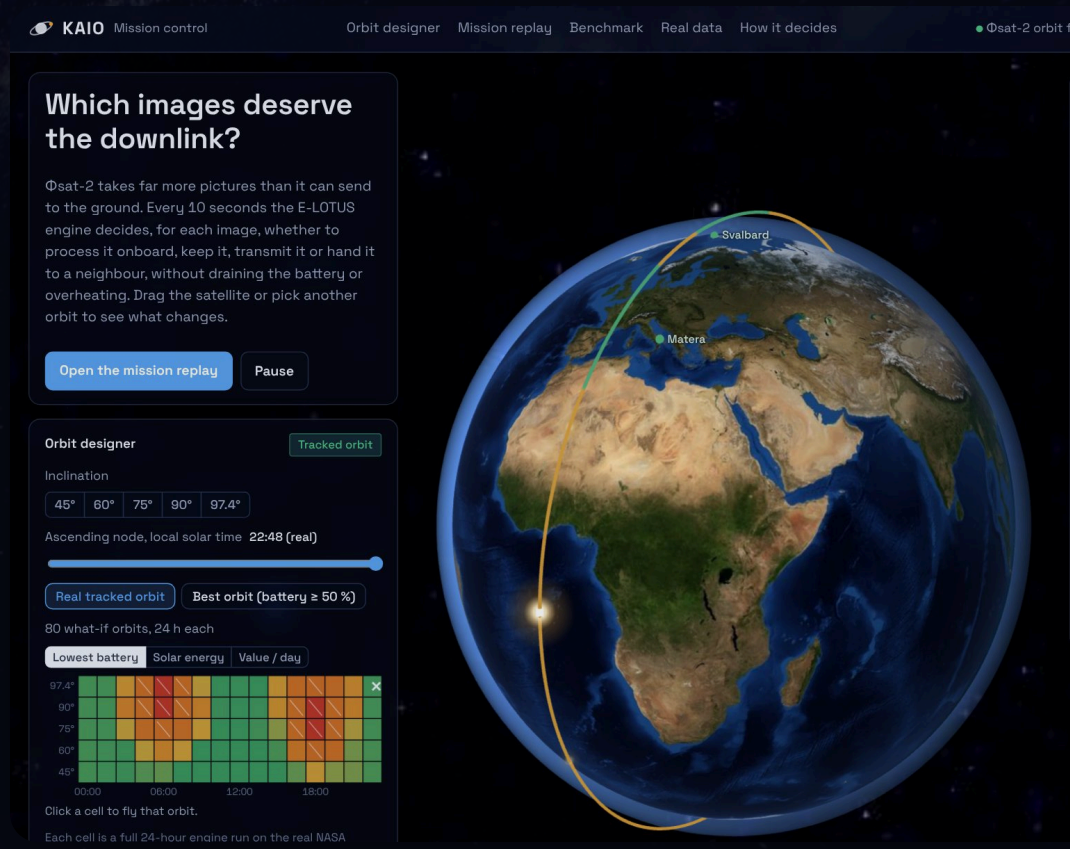
IASTAM 6.0 Technical Challenge · Track 1 · Problem 1 “Process or Transmit?”

KAIO

Which images deserve the downlink?

Onboard decisions for an Earth-observation satellite, flown on the real orbit of Φsat-2 with real NASA data.

Team KAIO · Nadhem Selmi · Mahmoud Amri



The problem: far more data than downlink

≈ 80 GB
of images produced per day

≈ 6 GB
can be sent down in 79 min of ground contact

35 min
of eclipse every 94-min orbit: no solar power

For every image, every 10 seconds, the satellite must choose, within its battery, heat, memory and contact limits:

Process
onboard AI: 50 MB → 1.5 MB, costs energy and heat

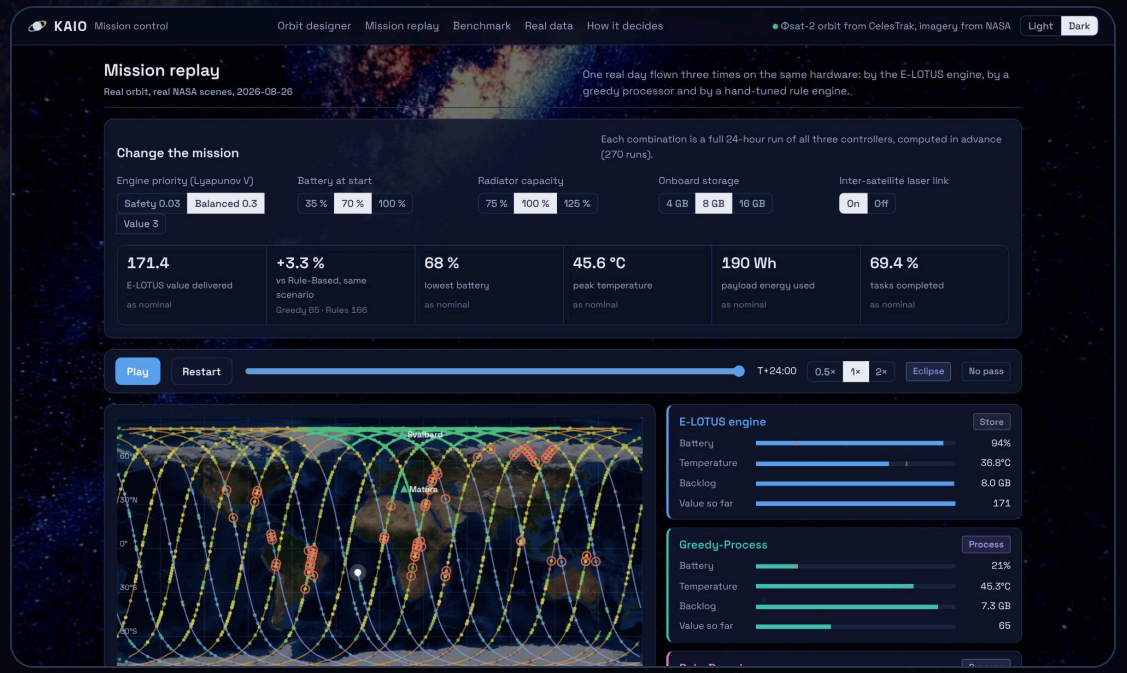
Keep
store for later, costs memory

Transmit
during a ground pass

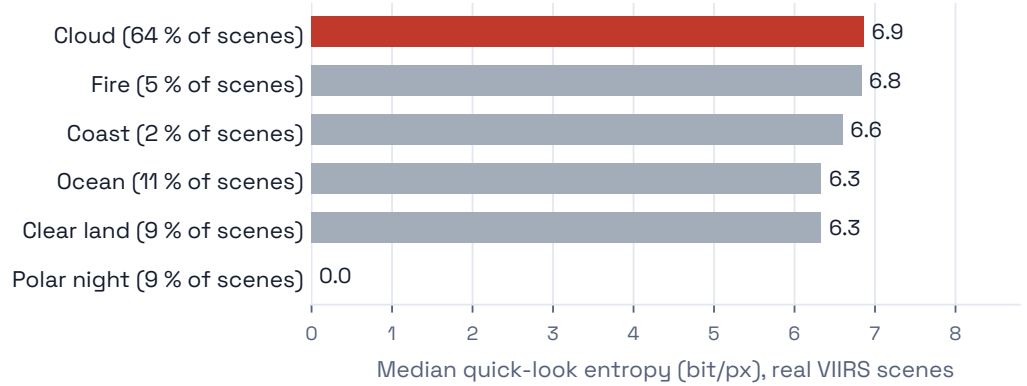
Offload
laser link to a neighbour in sunlight

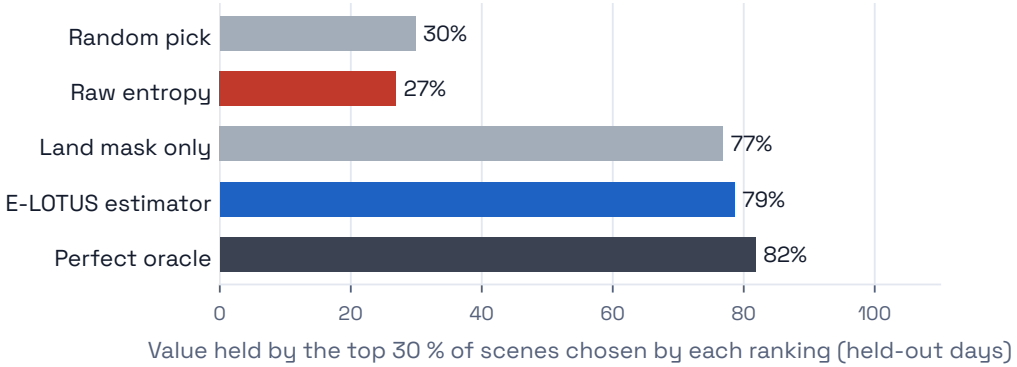
Real orbit, real NASA data

- **Orbit:** Φsat-2 (NORAD 60470) from its CelesTrak TLE, propagated with SGP4; RK4 + J2 cross-check within 25 km over 6 h
- **Ground stations:** KSAT Svalbard and ASI Matera, 10° elevation mask
- **Earth data:** NASA GIBS for 20 days under the real ground track: VIIRS images (day and night), MODIS clouds, FIRMS fires, land/water mask
- **57,600 real footprints** (313 km blocks). Every simulated picture is the real scene Φsat-2 flew over at that moment
- The spacecraft (battery, thermal, GPU, radio) is **simulated**; every assumption is in one file



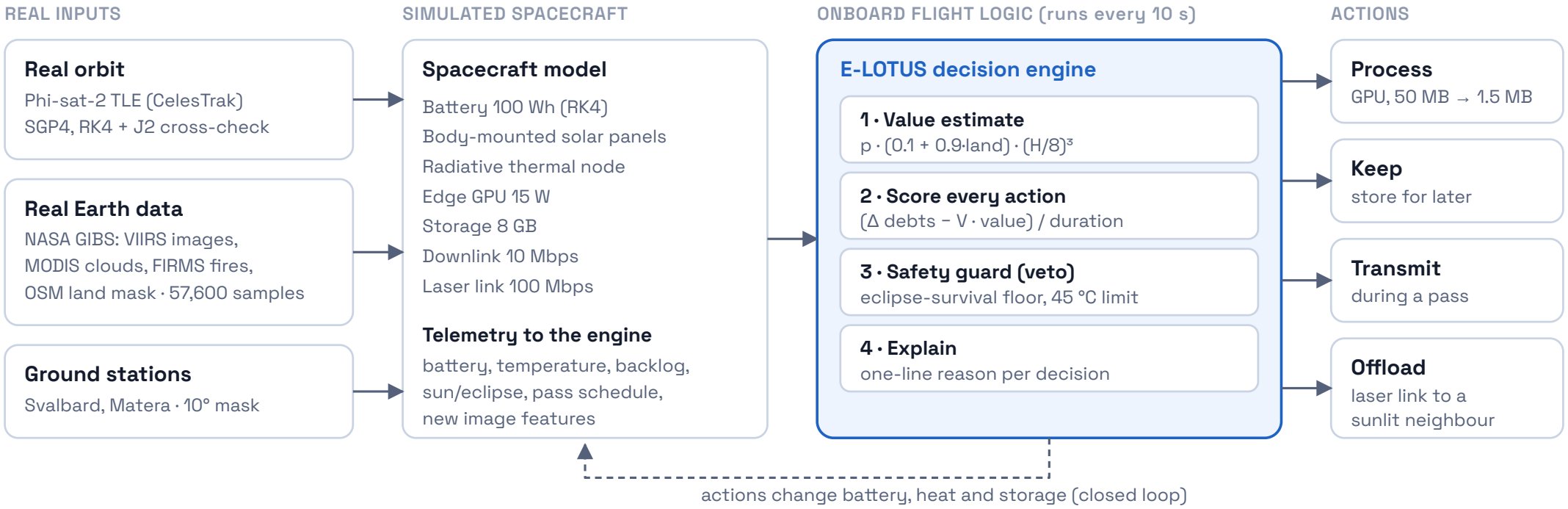
What real data taught us: clouds fool raw entropy





Clouds are the most textured scenes and 64 % of all scenes. Ranking images by texture keeps **27 %** of the value, worse than random (30 %). Multiplying by an onboard land mask gives **79 %**, close to a perfect oracle (82 %).

System architecture



SOFTWARE AROUND IT

Benchmark
8 scenarios × 6 strategies
× 20 real days, 95 % CI

Tests
18 automated checks (pytest)
incl. "never reads ground truth"

Mission-control dashboard
one offline HTML file,
three.js 3D Earth, 270 replays

Embedded target
Raspberry Pi: numpy + sgp4,
timing + live decision feed

How E-LOTUS decides, every 10 seconds

1

Value

texture × onboard land mask, × 4 for fire alerts

$p \cdot (0.1 + 0.9 \cdot land) \cdot (H/8)^3$

2

Score

growth of three debts (memory, battery below its eclipse floor, heat above 45 °C) minus V × value, per second

$(\Delta L - V \cdot U) / \tau$

3

Safety guard

independent veto, like flight-software FDIR: battery must survive the next eclipse, computer stays under 45 °C

fire alerts may use up to 60 °C

4

Explain

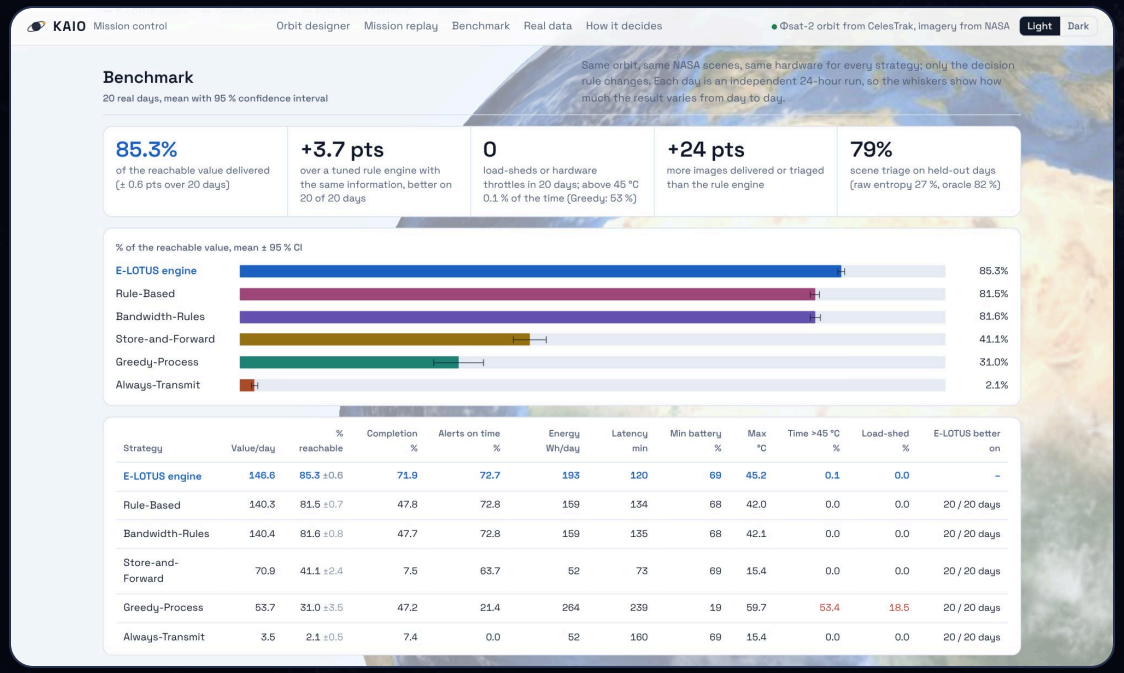
each decision logs a reason built only from onboard data

auditable by operators

> “Process now: est. value 0.68 (optical H 7.5, land 79 %). 20 MB raw → 0.6 MB product, next pass in 476 min; battery 69 Wh after (floor 20), 9.3 °C.”

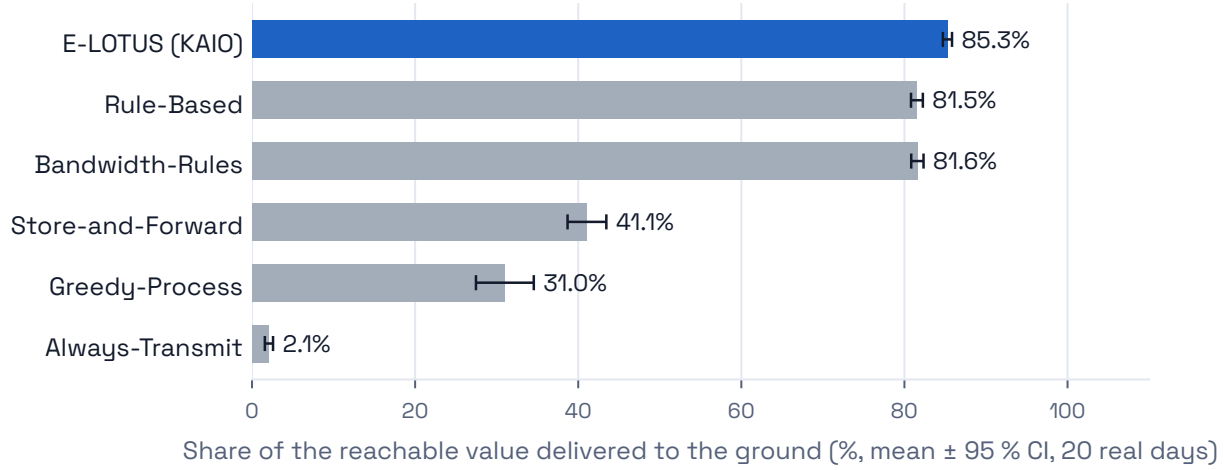
Drift-plus-penalty from Lyapunov optimisation, used as a principled scoring rule; all performance claims are measured, not proven.

Live demo



- **Mission-control dashboard:** 3D Earth with the real orbit, 80 what-if orbits, 270 pre-computed missions, one offline file
- **Raspberry Pi:** the same flight code runs live and prints each decision, its reason and its compute time
- Every number shown is regenerated by scripts from the benchmark

Results: 85.3 % of the reachable value



+3.7 pts
over a tuned rule engine with the same information; better on 20/20 days

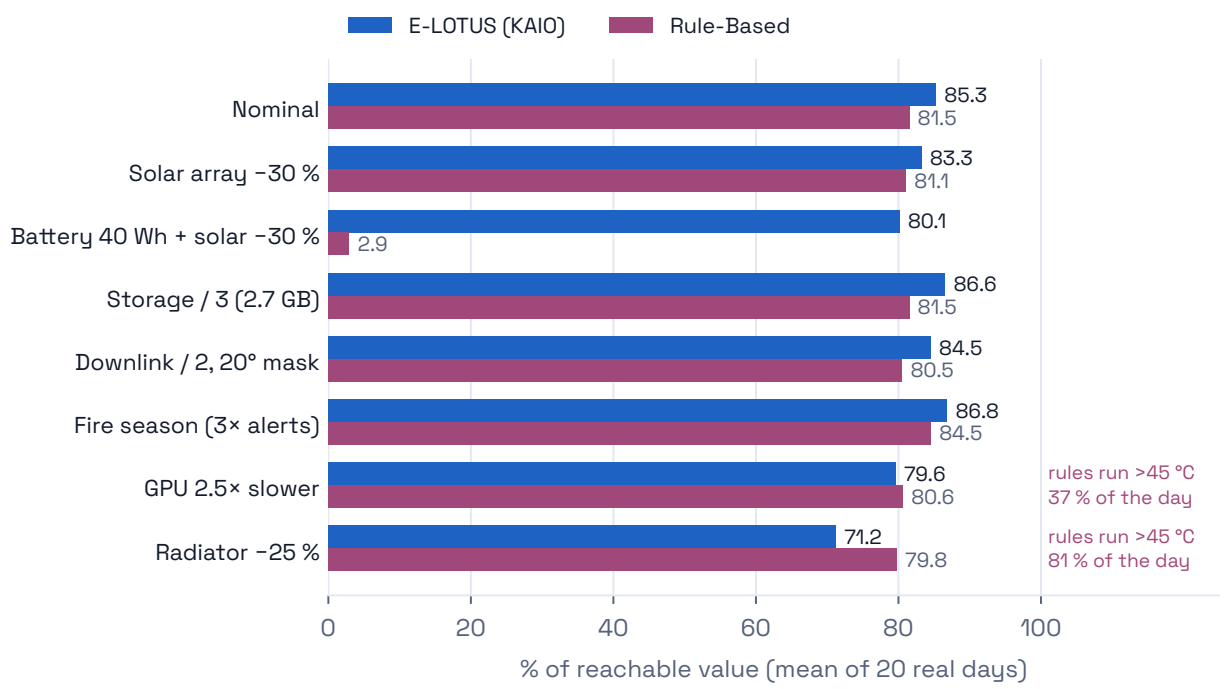
+24 pts
more images delivered or triaged (72 % vs 48 %)

0
load-sheds or throttles; battery never below 69 %

120 min
mean latency, vs 134 min for the rule engine

20 real days, each an independent 24-h run; mean ± 95 % confidence interval. Greedy processing drains the battery to 19 % and sheds the payload 19 % of the time.

Stress tests: where we win and where we don't



Wins 6/8. Weaker solar array, half the downlink, fire season, less storage: E-LOTUS wins every day.

Small battery: the rule engine's fixed threshold hits the shutdown floor (3 %). E-LOTUS adapts from the orbit (80 %).

Losses 2/8: slower GPU, weaker radiator. The rules only win by running above 45 °C for 37–81 % of the day. We respect the limit.

Feasible on real hardware

0.16 ms
median decision time (laptop (Pi run pending)); the satellite needs one every 10 s

0.002 %
of one CPU core over a full real day

2
libraries onboard: numpy and sgp4. No GPU, no training

Every input already exists on a satellite:

Battery and temperature
housekeeping sensors

Pass schedule
computed on the ground, uplinked

Land mask
static map, indexed by GNSS position

Thumbnail entropy
15 µs per image

Precedent: ESA's Φsat-1 already discarded cloudy images onboard. KAIO flies the real orbit of its successor.

Method and roadmap

How we built it

1. Two Phase-2 prototypes, merged into one tested package
2. Review found the old engine ran above 45 °C 10 % of the time; the safety guard fixed it
3. 18 automated tests, including “the engine never reads the ground truth”
4. 1,360 simulations: 8 scenarios × 6 strategies × 20 real days
5. Same code on a Raspberry Pi

Done	engine, safety guard, real-data simulation, benchmark, dashboard, embedded run
+3 mo	hardware-in-the-loop on a flight-like computer; imperfect forecasts and sensor noise
+6 mo	onboard cloud-screening network replacing the static land map; hindsight-optimal comparison
+12 mo	constellation coordination; partnership for a CubeSat flight test

KAIO decides which images deserve the downlink, explains why, and never breaks the spacecraft to do it.

85.3 %	20/20	0
of reachable value	days better than tuned rules	load-sheds or throttles

Thank you · questions welcome · github.com//kaio